



🔥 PERFORMANCE & COST OPTIMIZATION

Stop Waiting on Spark. Cut Latency by 75% & Master DP-700.

Starter Pools vs Custom Pools, High Concurrency session sharing, Native Velox Engine, and Environment item governance in Microsoft Fabric.

⚡ ~5s Starter Pools

🚀 4x Native Velox Engine

💰 Capacity Unit (CU) Saver

⚙️ SPARK ARCHITECTURE AT A GLANCE

Exam Domain 1

~5s

Starter Pool Allocation

Pre-warmed background nodes for instant execution

4x

Native C++ Velox Speedup

Vectorized execution bypassing JVM overhead

100%

DP-700 Exam Aligned

Real scenario decisions for workspace admins



POOL ARCHITECTURE

Starter Pools vs Custom Pools

Choosing the right compute pool type directly impacts job startup latency, scalability, and capacity costs.

⚡ STARTER POOLS (DEFAULT CONFIGURATION)

-  **STARTUP TIME**
~5 to 10 Seconds (Pre-warmed background nodes)
-  **MANAGEMENT**
Fully Managed by Fabric (Zero admin overhead)
-  **AUTO-SCALE**
Dynamic Node Allocation based on load
-  **BEST USED FOR**
Ad-hoc EDA, Quick Notebooks & Interactive Queries

⚙️ CUSTOM POOLS (DEDICATED SIZING)

-  **STARTUP TIME**
~2 to 3 Minutes (Dedicated Cold Start)
-  **NODE SIZING**
Small (4 vCore) to XX-Large (64 vCore / 512 GB)
-  **AUTO-PAUSE**
Configurable idle timeout (Default 2 mins)
-  **BEST USED FOR**
Heavy Batch ETL & Strict Memory Allocation



Exam Decision Key

If an exam scenario requires **instant startup time** with minimal setup, choose **Starter Pools**. If it requires **exact memory allocation**, **explicit executor boundaries**, or **specific node sizes**, choose **Custom Pools**.



CONCURRENCY MODEL

High Concurrency & Session Sharing

Share active Spark clusters across multiple notebooks to eliminate startup delays and maximize Capacity Unit (CU) efficiency.



Shared Spark Sessions

Multiple notebooks attach to an existing active session in seconds instead of spinning up isolated clusters.



CU Cost Savings

Prevents idle capacity consumption by reusing executor memory and CPU across concurrent engineering workloads.



User Security Isolation

Microsoft Fabric maintains user-level security boundaries and variable isolation across shared sessions.



Pipeline Reuse

Data Factory pipelines executing multiple notebook activities in parallel reuse the same session automatically.

✓ WHEN TO ENABLE HIGH CONCURRENCY MODE IN WORKSPACE SETTINGS

- **Interactive Team Development:** Multiple data engineers working in shared workspaces simultaneously.
- **Parallel Notebook Orchestration:** Pipelines that trigger 5+ PySpark Notebooks concurrently.
- **Capacity Throttling Mitigation:** Workspaces seeking to eliminate burst CU consumption and queuing delays.



ENGINE COMPONENTS

Fabric Spark Runtime Architecture

Understanding the pre-packaged Apache Spark, Delta Lake, and Python libraries provided out-of-the-box.

CORE ENGINE SPECIFICATIONS (FABRIC RUNTIME 1.3 / SPARK 3.5)



APACHE SPARK CORE
Apache Spark 3.5.0



DELTA LAKE PROTOCOL
Delta Lake 3.1.0 (Full Deletion Vectors & Liquid Clustering support)



PYTHON ENVIRONMENT
Python 3.10 / 3.11 with pre-installed Pandas, NumPy, Scikit-Learn, PySpark



VECTORIZED ENGINE
Native Execution Engine (Velox C++ acceleration layer)



Workspace Runtime Management

Workspace Admins can select the default runtime for all notebooks in **Workspace Settings** → **Data Engineering** → **Spark Settings**, ensuring version consistency across engineering teams.

[Data Engineering](#) > [DP-700](#) > [Native Velox](#)

PERFORMANCE BOOST

Native Execution Engine (Up to 4x Faster)

Replaces standard Java JVM execution with a C++ Velox vectorized processing engine directly inside Fabric.



C++ Vectorized Processing

Executes columnar operations directly on CPU SIMD instructions, bypassing JVM garbage collection overhead.



4x Speedup on Queries

Dramatically speeds up heavy aggregations, joins, string operations, and Delta table scans.

HOW TO ENABLE IN PYSPARK SESSION

```
# Enable Native Execution Engine in PySpark spark.conf.set("spark.native.enabled", "true") # Verify
engine execution status in Physical Plan df = spark.read.format("delta").load("Tables/sales")
df_filtered = df.filter(df["total_amount"] > 1000).groupBy("region").count() df_filtered.explain()
# Look for 'NativeScan' or 'Velox'
```



Exam Tip

When an exam scenario asks how to optimize long-running PySpark transformation jobs **without changing code or increasing pool size**, enabling the **Native Execution Engine** is the top answer.

[Data Engineering](#) > [DP-700](#) > [Environment Items](#)

LIBRARY GOVERNANCE

Environments vs Inline Libraries

Managing Python/PyPI dependencies efficiently across development workspaces.



Environment Items (Recommended)

Create reusable Environment items containing PyPI/Conda YAML specs. Attach to workspaces or specific notebooks for governance.



Inline %pip Commands

Use `%pip install library_name` at the top of a notebook for temporary ad-hoc testing. Installs only for the current session.

EXAMPLE: CONDA ENVIRONMENT YAML DEFINITION

```
name: fabric_data_engineering_env channels: - conda-forge dependencies: - python=3.10 - pip: - azure-storage-blob==12.19.0 - delta-spark==3.1.0
```



Best Practice

Avoid using inline `%pip install` in production scheduled pipelines because it adds installation latency on every pipeline run. Pre-publish dependencies into an **Environment Item**.



TELEMETRY & LOGS

Monitoring Spark Metrics & Telemetry

Track execution details, executor memory usage, and Capacity Unit (CU) consumption.



Spark UI Integration

Inspect Spark stages, tasks, DAG visualization, executor timelines, and memory spill directly in Fabric.



Capacity Metrics App

Monitor Spark background CU usage, identify throttling, and detect high-cost batch notebook runs.



Diagnostic Logs

Stream Spark driver and executor logs to Azure Log Analytics for centralized auditing and troubleshooting.

🎓 WHAT TO LOOK FOR IN SPARK UI (EXAM SCENARIOS)

- **Spill (Memory) & Spill (Disk):** Indicates memory pressure; increase executor size in Custom Pool.
- **Data Skew:** One task taking 90% of job duration; apply repartitioning or skew hints.
- **Stragglers:** Uneven partition distribution across executors.



EXAM DECISION GUIDE

Spark Pool Choice Matrix

Quick reference guide to select the right Spark pool type for DP-700 scenarios.

SCENARIO SELECTION CHEAT SHEET

SCENARIO REQUIREMENT

Interactive EDA notebook with instant startup requirement

Starter Pool

SCENARIO REQUIREMENT

Batch job requiring 128 GB memory & explicit 16-node limit

Custom Pool

SCENARIO REQUIREMENT

5 concurrent notebook activities in a single pipeline

High Concurrency

SCENARIO REQUIREMENT

Optimize long-running Delta scan without code changes

Native Engine

SCENARIO REQUIREMENT

Standardize Python libraries across 10 engineering notebooks

Environment Item



TIP #2 OF 30

Enable **Native Execution Engine** for Instant Boost

Before spending capacity units upgrading to larger Custom Pools, enable **Native Execution Engine** (`spark.native.enabled = true`). It uses C++ Velox processing to deliver up to **4x query speedup** at zero additional CU cost!

Zero CU Extra Cost

4x Query Acceleration

No Code Changes

Exam Favorite



DAY 2 COMPLETE

Day 2 — Done!

You now master **Starter Pools** vs **Custom Pools**, High Concurrency session sharing, Native Execution Engine, and Environment item management for DP-700.

Follow on LinkedIn for Day 3 →

Tomorrow: OneLake Settings & Access Control — Data Access Roles, RLS, OLS & Security Governance

ABOUT THE AUTHOR



Sayan Banerjee

Databricks Certified Data Engineer • Microsoft Fabric Data Engineer (DP-700) • Power BI Analyst (PL-300)

AI, BI & Analytics Consultant • Embee Software • Kolkata